



JS

Czym jest JavaScript?

Wprowadzenie do języka programowania, który napędza interaktywne strony internetowe



ROLA

Interaktywność



TYP

Skryptowy



ZASTOSOWANIE

Wieloplatformowe

JS

Dodawanie JS do HTML

Inline

JavaScript bezpośrednio w atrybutach HTML

```
<button  
onclick="alert('Cześć!')">Kliknij</button>
```

Internal

JavaScript wewnątrz znacznika
<script>

```
<script>  
console.log("Hello!");  
</script>
```

External

Oddzielny plik .js

```
<script src="app.js"></script>
```

Inline

- Szybkie rozwiązania
- Brak oddzielenia
- Trudne w utrzymaniu

Internal

- Łatwe dla prototypów
- Wszystko w jednym pliku
- Nieskalowalne

External

- Oddzielenie logiki
- Reużywalność kodu
- Łatwa konserwacja

Dobre praktyki

Używaj external scripts dla profesjonalnych projektów. Pliki JS umieszczaj na końcu <body> lub użyj atrybutu defer/async w <head>

Zmienne



let

```
let name = "Anna";
```

- Zmienna lokalna w bloku
- Można zmienić wartość
- Brak hoistingu (TDZ)



const

```
const PI = 3.14159;
```

- Stała - niezmiennalna
- Zasięg blokowy
- Wymagana wartość



var

```
var age = 25;
```

- Zasięg funkcyjny
- Hoisting (undefined)
- Styl przestarzały

↑ Hoisting

Przenoszenie deklaracji do góry zakresu

```
console.log(x); // undefined  
var x = 5;
```

≡ Zasady nazewnictwa

- camelCase: `userName`
- Zaczynać od litery, `_` \$
- Bez słów kluczowych
- Opisowa nazwa

👍 Dobre praktyki

Używaj `const` przez default, `let` gdy zmienna się zmienia. Unikaj `var` w nowym kodzie

🚫 Błędy

ReferenceError dla const/let przed deklaracją (TDZ). Zmienna const musi być zainicjowana

Typy danych - prymitywne

String

```
"Hello"
```

Tekst

Number

```
42, 3.14
```

Liczby całkowite i
zmiennoprzecinkowe

Boolean

```
true, false
```

Prawda / Fałsz

Null

```
null
```

Brak wartości (celowo)

Undefined

```
undefined
```

Niezdefiniowane

Symbol

```
Symbol("id")
```

Unikalne ID

BigInt

```
9007199254740991n
```

Duże liczby całkowite

typeof

Sprawdzanie typu:

```
typeof "string" → "string"
```

Niemutowalne

Typy prymitywne są niezmienniane.
Przekazywane przez wartość

Różnica null vs undefined

null: przypisany, brak wartości | undefined:
brak przypisania

Number

Obejmuje: `Infinity`, `NaN`, `0.1 + 0.2`

Dobre praktyki

Używaj `typeof` do rozróżniania typów. Pamiętaj, że `typeof null ≡ "object"` - to historyczny błąd JS

Typy danych - obiektowe

Object

Kolekcja par klucz-wartość

```
const person = {  
  name: "Anna",  
  age: 25  
};
```

mutable

przekazywane przez referencję

Array

Lista uporządkowana elementów

```
const colors = [  
  "red", "green", "blue"  
];
```

mutable

indeksowane od 0

typeof

Sprawdzanie typów:

```
typeof {} === "object"
```

```
typeof [] === "object"
```

```
typeof null === "object"
```

Dostęp do Object

Metody dostępu:

```
person.name
```

```
person["name"]
```

```
Object.keys(person)
```

Metody Array

Przykłady:

```
colors.push("yellow")
```

```
colors.map(c => c.toUpperCase())
```

```
colors.filter(c => c.length > 3)
```

Mutowalne

Obiekty są mutowalne. Przekazywane przez referencję, nie wartość

Uwaga: typeof null

`typeof null === "object"` - to historyczny błąd. Używaj `===` do sprawdzenia null

Kopiowanie

Używaj `{ ...obj }` lub `Object.assign()` do kopiowania obiektów

Operator

Arytmetyczne

```
a + b
```

```
a - b
```

```
a * b
```

```
a / b
```

```
a % b
```

```
a ** b
```

Porównania

```
a == b
```

```
a === b
```

```
a !== b
```

```
a >= b
```

```
a > b
```

```
a < b
```

Logiczne

```
a && b
```

```
a || b
```

```
!a
```

Przypisania

```
a = b
```

```
a += b
```

```
a -= b
```

```
a *= b
```

```
a /= b
```

```
a %= b
```

== vs ===

```
5 == "5" → true
```

```
5 === "5" → false
```

== konwertuje typy, === porównuje dokładnie

Krótkie obliczenia

```
a || b → a jeśli true
```

```
a && b → a jeśli false
```

Zwraca pierwszy value decydujący o wyniku

Inkrementacja

```
a++ → wartość, potem +1
```

```
++a → +1, potem wartość
```

Przydatne w pętlach

Dobra praktyka

Używaj `===` zamiast `==` dla uniknięcia błędów koercji typów

Kolejność działań

Priorytet: nawiasy → mnożenie → dodawanie → porównania → logiczne

Koercja typów i porównania

== (z konwersją)

Konwertuje typy przed porównaniem

```
5 == "5" → true
```

```
null == undefined → true
```

```
0 == false → true
```

```
"" == 0 → true
```

=== (bez konwersji)

Porównuje typ i wartość

```
5 === "5" → false
```

```
null === undefined → false
```

```
0 === false → false
```

```
"" === 0 → false
```

Truthy (prawda)

Wartości traktowane jako true

```
Boolean(1) → true
```

```
Boolean("text") → true
```

```
Boolean([]) → true
```

```
Boolean({}) → true
```

Falsy (fałsz)

Wartości traktowane jako false

```
Boolean(0) → false
```

```
Boolean("") → false
```

```
Boolean(null) → false
```

```
Boolean(undefined) → false
```

```
Boolean(NaN) → false
```

```
Boolean(false) → false
```

Dobra praktyka

Zawsze używaj `===` zamiast `==` dla uniknięcia nieoczekiwanych konwersji typów

Przykład praktyczny

Sprawdzanie czy tablica jest pusta: `arr.length === 0` zamiast `!arr.length`

Instrukcje warunkowe

→ if / else

```
if (score > 90) {  
  console.log("Doskonale");  
} else if (score > 70) {  
  console.log("Dobrze");  
} else {  
  console.log("Wymaga pracy");  
}
```

↔ switch

```
switch (day) {  
  case "Monday":  
    console.log("Poniedziałek");  
    break;  
  case "Tuesday":  
    console.log("Wtorek");  
    break;  
  default:  
    console.log("Inny dzień");  
}
```

↔ Operator trójargumentowy

Składnia:

```
warunek ? true : false
```

Przykład:

```
const grade = score > 90 ? "A" : "B";
```

✓ Dobre praktyki

Używaj operatora trójargumentowego dla prostych przypisań, if/else dla złożonych warunków

⚠ Pamiętaj o break

W switch używaj break aby przerwać przepływ, inaczej wszystkie case po spełnieniu zostaną wykonane

⊘ Falsy values

if (value) jest falszy dla: 0, "", null, undefined, NaN, false

Σ Zagnieżdżanie

Możesz zagnieżdżać instrukcje warunkowe, ale rozważ użycie && lub || dla prostszych przypadków

<> Skrócona forma

if (value) można skrócić do `value && action()`

for

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

while

```
let i = 0;  
while (i < 5) {  
  console.log(i++);  
}
```

do...while

```
let i = 0;  
do {  
  console.log(i++);  
} while (i < 5);
```

for...of

```
const arr = [1, 2, 3];  
for (const item of arr) {  
  console.log(item);  
}
```

for...in

```
const obj = {a: 1, b: 2};  
for (const key in obj) {  
  console.log(key, obj[key]);  
}
```

break

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;  
  console.log(i);  
}
```

continue

```
for (let i = 0; i < 5; i++) {  
  if (i === 2) continue;  
  console.log(i);  
}
```

Uwaga

for...in iteruje po kluczach obiektów i indeksach tablic

Praktyka

Używaj for...of dla tablic, for...in dla obiektów

Funkcje - podstawy

<> Declaration

```
function greet(name) {  
  return "Hello, " + name;  
}
```

Σ Expression

```
const greet = function(name) {  
  return "Hello, " + name;  
};
```

▶▶ Arrow Functions

Składnia podstawowa:

```
const add = (a, b) => {  
  return a + b;  
};
```

Skrócona forma:

```
const add = (a, b) => a + b;  
const square = x => x * x;
```

≡ Parametry domyślne

```
function greet(name = "Guest") {  
  return "Hello, " + name;  
}
```

... Rest Parameters

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}
```

← Return

Funkcja bez return zwraca undefined. Używaj return do zwracania wartości

i Hoisting

Function declarations są hoisted, ale expressions nie są

⚠ this w arrow

Arrow functions nie mają własnego this, dziedziczą z otoczenia

Funkcje - zaawansowane

Closures

```
function outer() {  
  let x = 10;  
  function inner() {  
    console.log(x);  
  }  
  return inner;  
}
```

Callbacks

```
function process(arr, callback) {  
  return arr.map(callback);  
}  
  
process([1, 2, 3], n => n * 2);
```

IIFE

```
(function() {  
  let privateVar = 10;  
  console.log(privateVar);  
})();
```

Scope

```
const global = "global";  
  
function func() {  
  let local = "local";  
  if (true) {  
    let block = "block";  
  }  
}
```

Closures

Funkcje pamiętają środowisko, w którym zostały utworzone. Dostęp do zmiennych z zakresu zewnętrznego

Callbacks

Funkcje przekazywane jako argumenty do innych funkcji i wywoływane później. Podstawa asynchroniczności

IIFE

Natychmiast wywoływana funkcja. Tworzy prywatny scope, zapobiegając zanieczyszczeniu global namespace

Global Scope

Zmienne dostępne wszędzie. Używaj const/let, unikaj var dla uniknięcia wycieków do global scope

Function Scope

Zmienne zadeklarowane w funkcji są dostępne tylko wewnątrz niej i funkcjach zagnieżdżonych

Block Scope

const i let w blokach {} mają zasięg blokowy. var ma zasięg funkcyjny

Tablice i metody

push / pop

```
const arr = [1, 2];  
arr.push(3); // [1, 2, 3]  
arr.pop(); // [1, 2]
```

map

```
const nums = [1, 2, 3];  
const doubled = nums.map(n => n * 2);  
// [2, 4, 6]
```

filter

```
const nums = [1, 2, 3, 4];  
const evens = nums.filter(n => n % 2 === 0);  
// [2, 4]
```

reduce

```
const nums = [1, 2, 3];  
const sum = nums.reduce((a, b) => a + b, 0);  
// 6
```

forEach

```
const nums = [1, 2, 3];  
nums.forEach(n => console.log(n));  
// 1, 2, 3
```

Metody łańcuchowe

```
const result = nums  
  .filter(n => n > 1)  
  .map(n => n * 2)  
  .reduce((a, b) => a + b, 0);
```

Mutujące vs nie

push, pop, shift, unshift zmieniają tablicę. map, filter, reduce zwracają nową

Shift / Unshift

shift usuwa z początku, unshift dodaje do początku tablicy

Find / Index

find zwraca pierwszy element, findIndex indeks, includes sprawdza obecność

Obiekty i JSON

Tworzenie obiektów

```
const person = {  
  name: "Anna",  
  age: 25,  
  greet() {  
    return "Hello";  
  }  
};
```

Dostęp do właściwości

```
person.name // "Anna"  
person["age"] // 25  
person.greet() // "Hello"
```

Metody obiektów

```
Object.keys(person) // ["name", "age"]  
Object.values(person) // ["Anna", 25]  
Object.entries(person) // [["name", "Anna"], ["age", 25]]
```

Modyfikacja

```
person.name = "Marek";  
person.email = "marek@example.com";  
delete person.age;
```

JSON.stringify()

```
const jsonStr = JSON.stringify(person);  
// {"name": "Anna", "age": 25}
```

JSON.parse()

```
const obj = JSON.parse(jsonStr);  
// {name: "Anna", age: 25}
```

Kopiowanie

Używaj `Object.assign()` lub `{...obj}` do kopiowania obiektów

`Object.freeze()`

Zamraża obiekt, zapobiegając modyfikacjom właściwości

`hasOwnProperty()`

Sprawdza, czy obiekt ma określoną właściwość

DOM - manipulacja

Wybieranie elementów

```
document.querySelector("#id");
document.querySelectorAll(".class");
document.getElementById("myId");
document.getElementsByClassName("cls");
```

Style

```
element.style.color = "red";
element.style.fontSize = "20px";
element.style.backgroundColor = "#fff";
```

Tworzenie

```
const div = document.createElement("div");
div.textContent = "Hello";
document.body.appendChild(div);
```

Usuwanie

```
element.remove();
parent.removeChild(child);
```

Atrybuty

```
element.setAttribute("id", "myId");
element.getAttribute("src");
element.removeAttribute("alt");
```

Zawartość tekstu

```
element.textContent = "Nowy tekst";
element.innerHTML = "<strong>Pogrubiony</strong>";
```

ClassList

```
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("highlight");
```

Good Practices

Używaj `querySelector` zamiast `getElementById` dla elastyczności. Unikaj `innerHTML` przy niezawodnych danych

Bezpieczeństwo

`innerHTML` może prowadzić do XSS. Używaj `textContent` lub `sanitize` przy danych użytkownika

DOM - zdarzenia

addEventListener

```
const button = document.querySelector("button");
button.addEventListener("click", function(event) {
  console.log("Clicked!");
});
```

Typowe zdarzenia

```
element.addEventListener("click", handler);
element.addEventListener("input", handler);
element.addEventListener("submit", handler);
element.addEventListener("load", handler);
```

Event Object

```
function handleClick(event) {
  event.target.style.color = "red";
  event.clientX, event.clientY;
  event.preventDefault();
}
```

Event Propagation

```
event.stopPropagation(); // zatrzymaj
event.stopImmediatePropagation();
event.stopPropagation() + event.preventDefault()
```

Bubbling

Zdarzenie propaguje w górę DOM

```
event.stopPropagation()
```

Capturing

Zdarzenie propaguje w dół DOM

```
element.addEventListener(event, handler, true)
```

preventDefault()

Zablokuj domyślne zachowanie

```
event.preventDefault()
```

Usunięcie nasłuchu

Używaj removeEventListener() aby usunąć nasłuch zdarzeń

Delegation

Przydziel jeden nasłuch do rodzica, obsługuj zdarzenia potomków

Asynchroniczność - podstawy

setTimeout

```
setTimeout(() => {  
  console.log("Za 2 sekundy");  
}, 2000);
```

setInterval

```
setInterval(() => {  
  console.log("Co sekundę");  
}, 1000);
```

Callbacks

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Dane z API");  
  }, 1000);  
}
```

Wyczyść timer

```
clearTimeout(timerId);  
clearInterval(intervalId);
```

Single-threaded

JavaScript wykonuje kod w jednym wątku, jeden proces naraz

Event Loop

Mechanizm zarządzający asynchronicznym kodem

Priority

Synchroniczny kod ma pierwszeństwo przed asynchronicznym

Callback Hell

Zagnieżdżone callbacks utrudniają utrzymanie kodu. Rozwiązanie: Promises/async-await

Dobre praktyki

Używaj Promises lub async-await zamiast zagnieżdżonych callbacks

Promises i async/await

⌚ Tworzenie Promise

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Sukces!");
  }, 1000);
});
```

↺ Stany Promise

```
Pending → Fulfilled
Pending → Rejected
```

⋯ .then() / .catch() / .finally()

```
promise
  .then(data => console.log(data))
  .catch(error => console.error(error))
  .finally(() => console.log("Koniec"));
```

↺ Async/Await

```
async function fetchData() {
  try {
    const data = await promise;
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

✅ Promise.all()

```
Promise.all([p1, p2, p3]).then()
```

▶▶ Promise.race()

```
Promise.race([p1, p2]).then()
```

🎯 Promise.any()

```
Promise.any([p1, p2]).then()
```

⚠️ Error Handling

Używaj try/catch z async/await, .catch() z Promises do obsługi błędów

✔ Best Practices

Async/await jest czytelniejsze niż zagnieżdżone .then(), używaj go w nowoczesnym kodzie

this keyword

Global Scope

```
console.log(this);  
// window (browser)  
// global (Node.js)
```

Metody Obiektu

```
const obj = {  
  name: "Test",  
  show() { console.log(this.name); }  
};  
obj.show(); // "Test"
```

Call / Apply / Bind

```
fn.call(thisArg, arg1, arg2);  
fn.apply(thisArg, [arg1, arg2]);  
fn.bind(thisArg);
```

Arrow Functions

```
const obj = {  
  name: "Test",  
  fn: () => console.log(this.name)  
};  
obj.fn(); // undefined
```

call()

Wywołuje funkcję z konkretnym this i argumentami

apply()

Podobnie jak call(), ale argumenty w tablicy

bind()

Tworzy nową funkcję z związanym this

Arrow Functions

Nie mają własnego this, dziedziczą z otoczenia. Nieużywane w metodach

Strict Mode

W strict mode global this jest undefined, nie window

Klasy i OOP

Tworzenie klasy

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  greet() {  
    console.log("Cześć " + this.name);  
  }  
}
```

Dziedziczenie

```
class Student extends Person {  
  constructor(name, major) {  
    super(name);  
    this.major = major;  
  }  
}
```

Metody statyczne

```
class Calculator {  
  static add(a, b) {  
    return a + b;  
  }  
}  
  
Calculator.add(2, 3); // 5
```

Pola prywatne

```
class Person {  
  #privateField = "tajne";  
  getSecret() {  
    return this.#privateField;  
  }  
}
```

Constructor

Inicjalizuje instancję klasy, ustawia właściwości początkowe

super()

Wywołuje konstruktor klasy nadrzędnej w dziedziczeniu

Metody

Funkcje zdefiniowane wewnątrz klasy, działają na instancjach

Best Practices

Używaj klas do struktury obiektowej, enkapsulacji i reużywalności kodu

Enkapsulacja

Pola prywatne z # chronią dane przed zewnętrznym dostępem

Moduły

↑ Export / Import

```
// utils.js
export const add = (a, b) => a + b;
export default function greet() {
  console.log("Hello");
}
```

↓ Import

```
// app.js
import { add } from "./utils.js";
import greet from "./utils.js";
import * as utils from "./utils.js";
```

Dynamiczne importy

```
button.addEventListener("click", async () => {
  const module = await import("./utils.js");
  module.default();
});
```

↔ CommonJS vs ESM

```
// CommonJS (Node.js)
const add = require("./utils");
module.exports = { add };

// ESM (Nowoczesny)
import { add } from "./utils.js";
```

📌 Nazwane importy

Import konkretnych elementów: `import { name }`

★ Default

Jeden domyślny export per plik: `export default`

🚫 Module Scope

Każdy moduł ma własny scope, bez zanieczyszczenia globalnego

✓ Best Practices

Używaj ESM (import/export) dla nowoczesnego kodu, CommonJS dla kompatybilności z Node.js

🛡️ Zalety

Lepsza organizacja kodu, unikanie konfliktów nazw, lazy loading

Wbudowane obiekty

Map

```
const map = new Map();
map.set("name", "Anna");
map.get("name"); // "Anna"
map.has("name"); // true
map.delete("name");
map.clear();
```

Set

```
const set = new Set([1, 2, 3, 3]);
set.add(4);
set.has(2); // true
set.delete(2);
set.forEach(v => console.log(v));
```

Date

```
const now = new Date();
now.getFullYear();
now.getMonth();
now.getDate();
now.toISOString();
```

WeakMap / WeakSet

```
const wm = new WeakMap();
const key = {};
wm.set(key, "data");
// klucze to tylko obiekty
// automatyczne czyszczenie
```

Iteracja

for...of, forEach, keys(), values(), entries()

Unikalność

Set gwarantuje unikalne wartości

Kolejność

Map zachowuje kolejność wstawiania

Best Practices

Używaj Map/Set zamiast Object/Array gdy potrzebujesz lepszej wydajności lub unikalnych wartości

Memory

WeakMap/WeakSet pozwalają na garbage collection kluczy, lepsze zarządzanie pamięcią

Obsługa błędów

⚠ Try / Catch / Finally

```
try {
  JSON.parse(invalidJSON);
} catch (error) {
  console.error(error.message);
} finally {
  console.log("Zawsze");
}
```

⚠ Throw Error

```
if (age < 0) {
  throw new Error("Wiek nie może być ujemny");
}
throw new TypeError("Niewłaściwy typ");
```

🐛 Typy Błędów

Error - ogólny błąd
TypeError - błąd typu
ReferenceError - brak zmiennej
SyntaxError - błąd składni

<> Custom Error

```
class CustomError extends Error {
  constructor(message) {
    super(message);
    this.name = "CustomError";
  }
}
```

📄 Stack Trace

error.stack pokazuje historię wywołań funkcji

🔄 Async Errors

Używaj try/catch wewnątrz async/await, .catch() z Promises

📄 Finally

Wykonywany zawsze, niezależnie od błędu

✅ Best Practices

Lapuj błędy tylko tam, gdzie możesz je obsłużyć. Nie ukrywaj błędów silent catch

⚠ Global Error Handler

window.onerror i unhandledrejection dla łapania nieobsłużonych błędów

DevTools i debugowanie

Console

```
console.log("Debug info");
console.error("Błąd");
console.table(obj);
console.time("timer");
console.group("Section");
```

Sources & Breakpoints

```
// Kliknij numer linii
// lub dodaj debugger;
let x = 10;
debugger;
console.log(x);
```

Watch Expressions

```
// Dodaj zmienne do watch
// Monitoruj wartości
// Sprawdzaj obiekty
// Śledź zmiany
```

Step-through Debugging

```
F10 - Step Over
F11 - Step Into
Shift+F11 - Step Out
F8 - Resume
```

Elements

Inspekcja DOM, modyfikacja stylów, śledzenie zdarzeń

Network

Analiza requestów, czasów ładowania, odpowiedzi

Application

Cookies, LocalStorage, IndexedDB, Cache

Best Practices

Używaj debugger w strategicznych miejscach, nie usuwaj console.log w produkcji, stosuj meaningful logs

Performance

Performance tab do analizy FPS, memory leaks, runtime performance

Event Loop

📦 Call Stack

Stos wywołań funkcji, LIFO

```
main() → func1() → func2()
```

```
[main, func1, func2]
```

📋 Macro Tasks

Task queue, timeout, events

```
setTimeout, setInterval,  
I/O, UI Events
```

```
[task1, task2, task3]
```

⚡ Microtasks

Higher priority queue

```
Promise.then,  
queueMicrotask
```

```
[promise1, promise2]
```

🔄 Event Loop Cycle

1

Wykonaj wszystkie microtasks



2

Przenieś task do stack



3

Wykonaj z stack



4

Powtórz cykl

! Priorytety

Microtasks > Macrotasks

```
Promise.then → setTimeout
```

<> Przykład

```
console.log(1);  
Promise.resolve().then(() ⇒ console.log(2));  
setTimeout(() ⇒ console.log(3), 0);  
console.log(4);  
// Output: 1, 4, 2, 3
```

✔ Single-threaded

JavaScript wykonuje kod w jednym wątku, ale asynchroniczność poprzez Event Loop

💡 Best Practices

Unikaj długich operacji synchronicznych, dziel kod na mniejsze taski, używaj microtasks priorytetowo

Fetch API

GET Request

```
fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

POST Request

```
fetch("/api/users", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({name: "Anna"})
})
```

Async / Await

```
async function fetchData() {
  try {
    const response = await fetch(url);
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Error Handling

```
if (!response.ok) {
  throw new Error(`HTTP error! status: ${response.status}`);
}
```

Response Methods

.json(), .text(), .blob(), .arrayBuffer()

HTTP Status Codes

response.ok, response.status, response.statusText

AbortController

Przerwanie requestu: controller.abort()

Best Practices

Zawsze sprawdzaj response.ok, używaj async/await dla czytelności, obsługuj błędy

Headers

Content-Type, Authorization, User-Agent w request headers

Podsumowanie i dobre praktyki

✓ Kluczowe Koncepty

- Single-threaded, asynchroniczny
- Event Loop i microtasks
- Closures i scope
- Promises i async/await
- Classes i OOP
- Moduły (ESM)

< > Dobre Praktyki

- Używaj const/let, unikaj var
- === zamiast ==
- Funkcje strzałowe
- Obsługa błędów (try/catch)
- Modularność
- Linters i formatters

📈 Narzędzia

- VS Code
- Chrome DevTools
- ESLint
- Prettier
- Jest (testy)
- Webpack/Vite

📖 Dokumentacja i Zasoby

- MDN Web Docs
- JavaScript.info
- developer.mozilla.org
- You Don't Know JS
- Eloquent JavaScript

🚀 Dalsze Kroki

- TypeScript
- React / Vue / Angular
- Node.js
- Testowanie (Jest, Vitest)
- CSS Frameworks (Tailwind)

💡 Rekomendacje dla Rozwoju

Praktyka jest kluczowa - pisz kod regularnie, buduj projekty, czytaj open source, korzystaj z społeczności. JavaScript ciągle ewoluuje - ucz się nowoczesnych featureków (ES2022+).

Praktyka

Projekty

Społeczność

Open Source

⚙️ Czysty Kod

Pisz czytelny, modularny kod z dobrą dokumentacją. Imiennictwa powinny być jednoznaczne i spójne.

👥 Społeczność

Dołącz do JavaScript Discord, GitHub, Stack Overflow. Ucz się od innych i dziel się wiedzą.